Lecture 2: Propositions, Proofs, and Logical Rules

Stefano M. Nicoletti

1 Lecture 2: Propositions, Proofs, and Logical Rules

1.1 Aim of the Lecture

This lecture recalls the role of the kernel and the typechecker, then introduces the way Lean reads propositions and proofs. The central point is the correspondence between propositions and types: a proposition `P : Prop` is a type, and a proof of `P` is a term inhabiting that type (Source: Theorem Proving in Lean 4, Propositions and Proofs).

We will work through minimal examples in order to observe, in the proof state, the introduction and elimination rules for the logical connectives and for a few specific constructs:

- implication `P → Q`;
- conjunction `P ∧ Q`;
- disjunction `P ∨ Q`;
- negation `¬P`;
- biconditional `P ↔ Q`;
- `False`;
- explicit classical principles.

1.2 Kernel and Typechecker

We have seen that Lean lets us build proofs with tactics, library support, and interactive interfaces. These tools help the user, but in the end the kernel checks the formal object that has been produced and verifies that it has the required type (Source: de Moura and Ullrich, Lean 4).

The typechecker checks whether a term has a certain type. In proofs, the expected type is the proposition that we want to prove. A theorem is accepted when Lean can check the proof term against the statement. From the system's point of view, proving means constructing a well-typed term.

This explains why the proof state is useful. The goal shows the type that we must inhabit. The assumptions show the terms already available in the context. A tactic is an interactive way to transform this situation until all goals are closed.

1.3 Propositions as Types

In Lean, `P : Prop` says that `P` is a proposition. If the context contains `hP : P`, then `hP` is a proof of `P`. It is not just an informal label: it is a formal object that can be used to build other proofs.

Implication is the most direct example. A term of type `P → Q` behaves like a function: it takes a proof of `P` and returns a proof of `Q`. If we have

```
hPQ : P → Q
hP : P
```

then `hPQ hP` is a proof of `Q`.

1.4 Introduction and Elimination Rules

For each logical connective we distinguish two questions.

- The introduction rule explains how to construct a proof of the compound proposition.
- The elimination rule explains how to use a proof of the compound proposition to obtain its components.

Looking at the goal means asking which introduction rule can construct it. Looking at the assumptions means asking which elimination rule can use them.

1.5 Implication

To introduce `P → Q`, we temporarily assume `P` and construct `Q`.

```
theorem lecture02_imp_intro (P Q : Prop) (hQ : Q) :
    P → Q := by
  intro hP
  exact hQ
```

The line `intro hP` introduces the temporary assumption `hP : P`. In this example the conclusion `Q` is already available as `hQ`, so we close the goal with `exact hQ`.

To eliminate `P → Q`, we apply the implication to a proof of `P`.

```
theorem lecture02_imp_elim (P Q : Prop) :
    (P → Q) → P → Q := by
  intro hPQ
  intro hP
  have hQ := hPQ hP
  exact hQ
```

Here `hPQ hP` is function application: from `P → Q` and `P` we obtain `Q`. This is modus ponens again.

1.6 Conjunction

A conjunction `P ∧ Q` contains both pieces of information. To introduce it, we must provide a proof of both components. In the next example they are given as assumptions in the statement.

```
theorem lecture02_and_intro (P Q : Prop) (hP : P) (hQ : Q) :
    P ∧ Q := by
  have hPAndQ := And.intro hP hQ
  exact hPAndQ
```

The same rule `And.intro` can be used as a tactic. The goal `P ∧ Q` splits into two subgoals, one for `P` and one for `Q`.

```
theorem lecture02_and_intro_with_apply (P Q : Prop) (hP : P) (hQ : Q) :
    P ∧ Q := by
  apply And.intro
  · exact hP
  · exact hQ
```

To eliminate a conjunction, we use its components and take the left or right conjunct.

```
theorem lecture02_and_elim_left (P Q : Prop) :
    P ∧ Q → P := by
  intro hPAndQ
  exact hPAndQ.left

theorem lecture02_and_elim_right (P Q : Prop) :
    P ∧ Q → Q := by
  intro hPAndQ
  exact hPAndQ.right
```

1.7 Disjunction

A disjunction `P ∨ Q` contains one of the two pieces of information. To introduce it, it is enough to provide one side, using `Or.inl` or `Or.inr`.

```
theorem lecture02_or_intro_left (P Q : Prop) :
    P → P ∨ Q := by
  intro hP
  exact Or.inl hP

theorem lecture02_or_intro_right (P Q : Prop) :
    Q → P ∨ Q := by
  intro hQ
  exact Or.inr hQ
```

To eliminate a disjunction, we reason by cases. If we have `P ∨ Q`, we do not generally know which side is available. To obtain a conclusion, we must produce it in both branches. In this example, we want to conclude `Q ∨ P` from `P ∨ Q`, so we consider the case where `P` is true and the case where `Q` is true.

```
theorem lecture02_or_elim (P Q : Prop) :
    P ∨ Q → Q ∨ P := by
  intro hPOrQ
  cases hPOrQ with
  | inl hP =>
      exact Or.inr hP
  | inr hQ =>
      exact Or.inl hQ
```

The same structure can be written with `Or.elim`.

```
theorem lecture02_or_elim_with_or_elim (P Q : Prop) :
    P ∨ Q → Q ∨ P := by
  intro hPOrQ
  apply Or.elim hPOrQ
  · intro hP
    exact Or.inr hP
  · intro hQ
    exact Or.inl hQ
```

1.8 False and Negation

`False` represents a contradictory state. If we have a proof of `False`, we can close any goal.

```
theorem lecture02_false_elim (P : Prop) :
    False → P := by
  intro hFalse
  exact False.elim hFalse
```

In Lean, negation is defined as implication to `False`: `¬P` means `P → False`. To eliminate a negation, we apply it to a positive proof of `P`.

```
theorem lecture02_not_elim (P : Prop) :
    P → ¬P → False := by
  intro hP
  intro hNonP
  exact hNonP hP
```

To introduce `¬P`, we temporarily assume `P` and derive `False`. The next example is the reasoning pattern of modus tollens. The line `intro hP` has this role: to prove `¬P`, assume `P` and derive `False`. Observe how the goal changes.

```
theorem lecture02_not_intro (P Q : Prop) :
    (P → Q) → ¬Q → ¬P := by
  intro hPQ
  intro hNonQ
  intro hP
  have hQ := hPQ hP
  exact hNonQ hQ
```

Here is an instantiated version: if drinking wine implies being drunk, and we are not drunk, then we did not drink wine. The line `intro hDrinkWine` plays the same role as `intro hP`.

```
theorem lecture02_wine_modus_tollens
    (DrinkWine Drunk : Prop) :
    ((DrinkWine → Drunk) ∧ ¬Drunk) → ¬DrinkWine := by
  intro assumption
  have hDrinkDrunk := assumption.left
  have hNotDrunk := assumption.right
  intro hDrinkWine
  have hDrunk := hDrinkDrunk hDrinkWine
  exact hNotDrunk hDrunk
```

1.9 Biconditional

A biconditional `P ↔ Q` contains two implications: one from `P` to `Q` and one from `Q` to `P`. To introduce it, we must construct both directions and use `Iff.intro`.

```
theorem lecture02_iff_intro (P Q : Prop) :
    (P → Q) → (Q → P) → (P ↔ Q) := by
  intro hPQ
  intro hQP
  apply Iff.intro
  · exact hPQ
  · exact hQP
```

To eliminate a biconditional, we use one of the two directions. In Lean, `Iff.mp` is the left-to-right direction, while `Iff.mpr` is the right-to-left direction.

```
theorem lecture02_iff_elim_left (P Q : Prop) :
    (P ↔ Q) → P → Q := by
  intro hIff
  intro hP
  exact Iff.mp hIff hP

theorem lecture02_iff_elim_right (P Q : Prop) :
    (P ↔ Q) → Q → P := by
  intro hIff
  intro hQ
  exact Iff.mpr hIff hQ
```

1.10 Intuitionistic Foundations

Lean's basic logic is constructive, or intuitionistic. This means that a proof must construct, step by step, what the goal requires.

Some familiar principles of classical mathematics are not available automatically in this constructive base:

- excluded middle, `P ∨ ¬P`;

- double-negation elimination, `¬¬P → P`;
- classical proof by contradiction.

These principles can be used in Lean, but they must be invoked explicitly through the classical part of the logical library.

1.11 Contraposition

The direction

```
(P → Q) → (¬Q → ¬P)
```

is constructive. If we have `P → Q` and `¬Q`, then to prove `¬P` we assume `P`, obtain `Q`, and apply `¬Q`.

```
theorem lecture02_contraposition_forward (P Q : Prop) :
    (P → Q) → (¬Q → ¬P) := by
  intro hPQ
  intro hNotQ
  intro hP
  have hQ := hPQ hP
  exact hNotQ hQ
```

The reverse direction,

```
(¬Q → ¬P) → (P → Q)
```

requires classical reasoning. From `¬Q → ¬P` and `P`, we do not know how to construct a proof of `Q` directly. We therefore use excluded middle on `Q` and reason by cases with `cases Classical.em Q with`.

```
theorem lecture02_contraposition_classical (P Q : Prop) :
    (¬Q → ¬P) → (P → Q) := by
  intro hContrap
  intro hP
  cases Classical.em Q with
  | inl hQ =>
      exact hQ
  | inr hNotQ =>
      have hNotP := hContrap hNotQ
      exfalso
      exact hNotP hP
```

We can combine the two directions into a biconditional.

```
theorem lecture02_contraposition_full (P Q : Prop) :
    (P → Q) ↔ (¬Q → ¬P) := by
  apply Iff.intro
  · exact lecture02_contraposition_forward P Q
  · exact lecture02_contraposition_classical P Q
```

The theoretical point is that the full biconditional uses one constructive direction and one classical direction. For the classical direction, we must specify the use of classical constructs such as excluded middle.

1.12 Proof by Contradiction

Excluded middle is available in Lean as `Classical.em P`, which produces a proof of `P ∨ ¬P`. This allows us to reason by cases on an arbitrary proposition. We can use it in classical proof by contradiction: assuming `¬P` leads to `False`, so classically we conclude `P`.

```
theorem lecture02_proof_by_contradiction (P : Prop) :
    (¬P → False) → P := by
  intro hContradiction
  cases Classical.em P with
  | inl hP =>
      exact hP
  | inr hNotP =>
      exfalso
      exact hContradiction hNotP
```

Lean also provides the dedicated classical principle `Classical.byContradiction`.

```
theorem lecture02_by_contradiction_classical (P : Prop) :
    (¬P → False) → P := by
  intro hContradiction
  exact Classical.byContradiction hContradiction
```

The theoretical note is important. This step is not the same as constructive negation. The constructive rule `P → ¬P → False` says that `P` and `¬P` are incompatible and generate a contradiction. The classical conclusion by contradiction `(¬P → False) → P`, instead, eliminates a double negation and constructs `P`.

1.13 Argumentation Examples

After these minimal rules, we return to natural-language arguments with more awareness.

```
-- Natural language: if we have an assumption, if a consequence follows from
-- the assumption, and if a conclusion follows from the consequence, then we
-- have the conclusion.
example (Assumption Consequence Conclusion : Prop) :
    (Assumption ∧ (Assumption → Consequence)) ∧
      (Consequence → Conclusion) →
    Conclusion := by
  intro hArgument
  have hFirstStep := hArgument.left
  have hAssumption := hFirstStep.left
  have hStep1 := hFirstStep.right
  have hStep2 := hArgument.right
  have hConsequence := hStep1 hAssumption
  have hConclusion := hStep2 hConsequence
  exact hConclusion
```

The second example uses a disjunction.

```
-- Natural language: if it rains or it snows, and in each of the two cases I
-- take the umbrella, then I take the umbrella.
example (Rains Snows TakeUmbrella : Prop) :
    ((Rains → TakeUmbrella) ∧
      (Snows → TakeUmbrella)) ∧
    (Rains ∨ Snows) →
    TakeUmbrella := by
  intro hArgument
  have hRules := hArgument.left
  have hRainsOrSnows := hArgument.right
  have hRainsUmbrella := hRules.left
  have hSnowsUmbrella := hRules.right
  -- We do not know which side of the ∨`` is valid, so we consider both cases.
  cases hRainsOrSnows with
  | inl hRains =>
      exact hRainsUmbrella hRains
  | inr hSnows =>
      exact hSnowsUmbrella hSnows
```

The version with `Or.elim` makes the abstract structure of the rule explicit: from a proof of `A ∨ B`, a function `A → C`, and a function `B → C`, we obtain `C`.

```
-- The same proof, using `Or.elim` directly.
example (Rains Snows TakeUmbrella : Prop) :
    ((Rains → TakeUmbrella) ∧
      (Snows → TakeUmbrella)) ∧
    (Rains ∨ Snows) →
    TakeUmbrella := by
  intro hArgument
  have hRules := hArgument.left
  have hRainsOrSnows := hArgument.right
  have hRainsUmbrella := hRules.left
  have hSnowsUmbrella := hRules.right
  apply Or.elim hRainsOrSnows
  · intro hRains
    exact hRainsUmbrella hRains
  · intro hSnows
    exact hSnowsUmbrella hSnows
```

One final example:

```
-- Natural language: if the source is authentic or the data are coherent, and
-- each of the two cases supports the thesis, and if a supported thesis makes
-- the conclusion plausible, then the conclusion is plausible.
example (AuthenticSource CoherentData SupportedThesis PlausibleConclusion :
    Prop) :
    ((AuthenticSource ∨ CoherentData) ∧
      ((AuthenticSource → SupportedThesis) ∧
        (CoherentData → SupportedThesis))) ∧
    (SupportedThesis → PlausibleConclusion) →
    PlausibleConclusion := by
```

```
  intro hArgument
  have hFirstStep := hArgument.left
  have hSourceOrData := hFirstStep.left
  have hSupportRules := hFirstStep.right
  have hSourceSupports := hSupportRules.left
  have hDataSupport := hSupportRules.right
  have hSupportConcludes := hArgument.right
  have hThesis :=
    Or.elim hSourceOrData hSourceSupports hDataSupport
  have hConclusion := hSupportConcludes hThesis
  exact hConclusion
```

1.14 Tactics, Commands, and Shortcuts

In this lecture we mostly used tactics corresponding to the introduction and elimination rules for the connectives.

- `intro`: introduces a temporary assumption when the goal is an implication or a negation;
- `exact`: closes the goal by providing a term of the required type;
- `have`: introduces an intermediate result into the context;
- `apply`: applies a rule, theorem, or constructor to the current goal;
- `cases`: distinguishes the cases of a disjunction;
- `exfalso`: changes the current goal to `False` when we want to close it by contradiction.

We also used some explicit constructors and eliminators:

- `And.intro`, `.left`, `.right` for conjunction;
- `Or.inl`, `Or.inr`, `Or.elim` for disjunction;
- `False.elim` for using a contradiction;
- `Iff.intro`, `Iff.mp`, `Iff.mpr` for the biconditional;
- `Classical.em` for excluded middle;
- `Classical.byContradiction` for classical proof by contradiction.

1.15 Exercises and Solutions

The exercise and solution files (`Exercises.lean` and `Solutions.lean`) are available on the course website.

1.16 Sources and Further Reading

- Jeremy Avigad, Leonardo de Moura, Soonho Kong, Sebastian Ullrich, *Theorem Proving in Lean 4*, chapter "Propositions and Proofs": https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/.

- Leonardo de Moura and Sebastian Ullrich, ”The Lean 4 Theorem Prover and Programming Language”: https://lean-lang.org/papers/lean4.pdf.